

Monitorowanie stanu powietrza

Założmy, że tysiące punktów pomiaru z całej Europy wysyła średnio co kilka minut informacje o aktualnym stanie powietrza. Dane te są odbierane przez warstwę serwisów, za którą odpowiada Twój zespół. Następnie, dla każdego ze wskaźników zanieczyszczenia powietrza obliczane są wartości średnie/maksymalne, a rezultat obliczeń prezentowany jest na interfejsie użytkownika.

Jeden z udostępnianych endpointów GET `/api/country/{countryId}/cities/stats/24h` zwraca dane w następującym formacie:

```
[
  { "city": "Gdańsk", "maxNO2": "18.67", "maxCO": "11.45", "maxPM10": "0.29" },
  { "city": "Jaworzno", "maxNO2": "41.56", "maxCO": "31.23", "maxPM10": "1.28" },
  { "city": "Kraków", "maxNO2": "22.31", "maxCO": "12.46", "maxPM10": "0.3" },
  { "city": "Poznań", "maxNO2": "26.78", "maxCO": "15.89", "maxPM10": "0.48" },
  { "city": "Ruda Śląska", "maxNO2": "38.92", "maxCO": "28.54", "maxPM10": "1.12" },
  { "city": "Warszawa", "maxNO2": "16.34", "maxCO": "9.87", "maxPM10": "0.22" },
  { "city": "Wrocław", "maxNO2": "23.45", "maxCO": "13.21", "maxPM10": "0.35" }
]
```

Gdzie:

- `city` - miejscowość, w której znajduje się stacja wykonująca pomiary
- `maxPM10` - wskaźnik pyłu zawieszonego PM10
- `maxCO` - wskaźnik tlenku węgla
- `maxNO2` - wskaźnik dwutlenku azotu

Bazując na API zdefiniowanym powyżej, napisz interfejs użytkownika dla osób zajmujących się analizą zanieczyszczenia powietrza.

Wymagania

1. Głównym elementem interfejsu powinna być tabelka zawierająca dane ze wszystkich miast, zasilana po wybraniu roku pomiarów oraz konkretnego kraju z listy dostępnych (do decyzji przy użyciu jakich kontrolek).
 - Dane powinny być domyślnie posortowane alfabetycznie po nazwie miasta, jednak powinno być możliwe sortowanie po dowolnej kolumnie.
 - Powinno być możliwe filtrowanie danych po mieście, wpisując dowolny ciąg znaków.
 - Dane powinny być automatycznie odświeżane co 20 sekund (polling). Nie ma potrzeby implementacji komunikacji real-time (np. WebSocket lub SSE). Zadbaj o to, aby interfejs zachowywał się przewidywalnie podczas automatycznych odświeżeń (np. nie gubił aktualnych filtrów, sortowania ani kontekstu użytkownika).
 - Przyjmij, że API może odpowiadać z opóźnieniem, a aplikacja powinna poprawnie obsługiwać stany ładowania danych.
 - Przyjmij również, że wywołanie API może zakończyć się błędem - wymagana jest

obsługa negative path, prezentacji stanu błędu oraz możliwości ponowienia żądania (retry).

- Odpowiedź API może pominąć rekord dla danego miasta, jeśli nastąpi awaria systemu pomiarowego. Jeśli wystąpi awaria jednego z czujników, to wartości `maxNO2`, `maxCO` lub `maxPM10` mogą przyjąć wartość `null`, np.:

```
{ "city": "Radom", "maxNO2": null, "maxCO": "13.21", "maxPM10": "0.35" }
```

2. Nad tabelką powinien być prezentowany wykres kolumnowy pokazujący wartości maksymalne NO2 dla każdego miasta. Wykres powinien być aktualizowany dynamicznie przy każdej zmianie danych, zakresu czasowego lub kraju. Opcjonalnie można dodać możliwość wyboru, które wskaźniki mają być prezentowane na wykresie.
3. Dodatkowo użytkownik powinien móc przejść do notatek na temat dowolnego miasta poprzez URL `/api/cities/{cityId}/notes`, gdzie zobaczy listę notatek dotyczących danego miasta, w kolejności od najnowszej do najstarszej biorąc pod uwagę datę utworzenia.
 - Na górze strony powinien znajdować się przycisk "Nowa notatka", którego kliknięcie powoduje zaprezentowanie niewielkiego okna typu pop-up, pozwalającego na wprowadzenie tytułu oraz treści notatki. Okno powinno zawierać przycisk "Zapisz", który zapisze notatkę w bazie danych.
 - Każda notatka na liście powinna zawierać:
 - tytuł, datę dodania, datę modyfikacji
 - przycisk "Szczegóły" - otwierający okno typu pop-up ze wszystkimi informacjami, które widoczne są w wierszu listy + treść notatki
 - przycisk "Edytuj" - po kliknięciu w przycisk użytkownik powinien zobaczyć okno typu pop-up, na którym możliwa jest edycja wyłącznie treści notatki. Klikając w przycisk "Zapisz" powinna zostać zaktualizowana treść oraz data modyfikacji notatki
 - Widok notatek powinien uwzględniać obsługę stanów ładowania, oraz błędów zapisu i odczytu.

-
- Preferowane technologie: React, TypeScript
 - Nie ma wymogu implementacji części backendowej. Dane mogą być mockowane w dowolny sposób, np. przy użyciu MSW, json-servera, lokalnego mock service lub innej wybranej techniki.
 - Dokładna architektura aplikacji jest Twoją decyzją
 - Dokładny wygląd interfejsu użytkownika jest Twoją decyzją
 - Do kodu dołącz plik README z krótkim opisem rozwiązania